ELSA 3.9
TICKET API GUIDE

Requests, responses and approval / rejection journeys

Expense Claim | Tested application | 25 October 2026

This guide explains how to use the tested application through Swagger and Elsa APIs. It covers login, workflow publication, ticket submission, task claiming, approval, rejection, correction and history. Configuration internals are excluded.

The app has been restored to the configurationJson-backed workflow that supports Approve, Reject and Resubmit. The complete publication request is supplied as a companion JSON file; you do not need to edit it to follow this guide.

Evidence: 148 captured HTTP exchanges from an isolated preview instance. Response excerpts use actual captured values; JWTs and sample login passwords are redacted. API examples use localhost:5000; the capture run used port 5007.

Step	API journey
1	Choose your user and understand the stage route
2	Login, publish and execute
3	Find and claim a task
4	Approve and check the next stage
5	Reject to A, requester or terminal Rejected
6	Correct and resubmit
7	Read the full journey
8	Handle errors and retries
9	Review tested results

1. Users and the ticket journey

Run the application and open http://localhost:5000/swagger. Use the account whose role matches the next action. The verified run uses in-memory preview mode; restarting preview clears definitions and tickets. This guide does not claim PostgreSQL restart persistence.

Account	Password / purpose
admin	password / administrative overview
configurator	Password123! / publish workflow
requester	Password123! / submit and correct claims
alice, amy	Password123! / stage A
bella, bob	Password123! / stage B
clara	Password123! / high-value stage C
dan	Password123! / final stage D
supervisor	Password123! / native supervision permissions

These are local research credentials. All user IDs equal the sample usernames. A and B are buckets: one eligible member claims the task. C belongs to Clara and D belongs to Dan. B is not automatic round robin.

```
A --Approve--> B --amount > 5000--> C --Approve--> D
                 --amount <= 5000----------------> D
D --Approve--> Completed

A --Reject--> requester --Resubmit--> A
B --Reject--> A (new task)
C --Reject--> Rejected (terminal)
D --Reject--> requester --Resubmit--> D
```

A claim of exactly 5000 skips C. Rejected A and D return for correction to the same stage. Changes made after D returns a claim do not replay B or require a new C approval.

2. Login, publish and execute

2.1 Login and Swagger Authorize

```
POST http://localhost:5000/elsa/api/identity/login
Content-Type: application/json

{"password": "[development sample password]", "username": "configurator"}
```

Captured HTTP 200. Response excerpt; fields outside the excerpt are omitted. IDs and timestamps are from an isolated test run.

```
{"isAuthenticated": true, "accessToken": "[REDACTED]", "refreshToken": "[REDACTED]"}
```

Use Password123! for the sample configurator (the capture redacts it). Copy accessToken into the Swagger JWT bearer Authorize field. Use each task owner's token when acting on that task. A successful HTTP response without an accessToken does not prove authentication.

2.2 Publish the complete definition

```
POST http://localhost:5000/elsa/api/workflow-definitions
Authorization: Bearer <configurator-accessToken>
Content-Type: application/json
```

Captured HTTP 200. Response excerpt; fields outside the excerpt are omitted. IDs and timestamps are from an isolated test run.

```
{
  "workflowDefinition": {
    "id": "4947fa042723c007",
    "definitionId": "expense-claim-v1",
    "name": "Expense Claim",
    "version": 1,
    "isPublished": true
  }
}
```

Request body: the complete companion definition JSON, including model.root.configurationJson.expression.value. Publishing uses the configurator token. The activity validates the business configuration when executed; publishing alone does not guarantee the configuration is valid.

2.3 Submit a claim as requester

```
POST http://localhost:5000/elsa/api/workflow-definitions/expense-claim-v1/execute
Authorization: Bearer <requester-accessToken>
Content-Type: application/json

{
  "input": {
    "requesterId": "requester",
    "lastName": "Shah",
    "claimAmount": 4000,
    "firstName": "Pranav"
  },
  "correlationId": "DOC-approved-4000-b201cc"
}
```

Captured HTTP 200. Response excerpt; fields outside the excerpt are omitted. IDs and timestamps are from an isolated test run.

```
{
  "workflowState": {
    "id": "aa7aaaa3c939dc4f",
    "definitionId": "expense-claim-v1",
    "correlationId": "DOC-approved-4000-b201cc",
    "status": "Running",
    "subStatus": "Suspended"
  }
}
```

This 4000 claim follows A-B-D. Change claimAmount to 6000 to include Clara at C. The find, claim and first approval examples below refer to this same captured 4000 workflow. Replace captured IDs with IDs from your own server.

correlationId is your business reference. requesterId is the stable identity ID, checked against the authenticated requester (admin may submit for another requester). claimAmount is a JSON number. Save workflowState.id: it is the workflow instance ID used for task filtering and history. correlationId is not a submission idempotency key; repeated execute calls may create another instance.

3. Find a task and claim ownership

3.1 Administrative overview

```
GET http://localhost:5000/elsa/api/user-tasks?scope=all&workflowInstanceId=aa7aaaa3c939dc4f&include
TotalCount=true
Authorization: Bearer <admin-accessToken>
```

Captured HTTP 200. Response excerpt; fields outside the excerpt are omitted. IDs and timestamps are from an isolated test run.

```
{
  "items": [
    {
      "id": "2c67725fd185baf8",
      "title": "A: Initial review (Alice/Amy bucket)",
      "status": "Available",
      "workflowInstanceId": "aa7aaaa3c939dc4f",
      "revision": 1
    }
  ],
  "totalCount": 1
}
```

scope=all requires elevated access. Ordinary approvers use GET /elsa/api/user-tasks?scope=available for claimable work and scope=assigned for owned work, adding workflowInstanceId as needed. Only eligible identities see their work; a valid token from the wrong member may legitimately return an empty list.

3.2 Claim A as Alice

```
POST http://localhost:5000/elsa/api/user-tasks/2c67725fd185baf8/claim
Authorization: Bearer <alice-accessToken>
Content-Type: application/json

{"operationId": "8793f14b-d80d-4aaf-bbac-9d6bffebefac", "expectedRevision": 1}
```

Captured HTTP 200. Response excerpt; fields outside the excerpt are omitted. IDs and timestamps are from an isolated test run.

```
{
  "operationId": "8793f14b-d80d-4aaf-bbac-9d6bffebefac",
  "status": "completed",
  "revision": 2,
  "task": {
    "id": "2c67725fd185baf8",
    "status": "Assigned",
    "revision": 2,
    "assignee": {"id": "alice"}
  }
}
```

expectedRevision is the latest revision from this task, not a workflow or definition version. This sample starts A at revision 1; claiming returns revision 2. Read the task again before completion if anything changed.

operationId is a caller-generated UUID identifying one command attempt. Keep the same ID and identical payload when retrying an uncertain request. Generate a new ID for a different command; do not reuse the claim ID for completion.

A/B buckets require a claim before completion. C and D are assigned directly. Use GET /elsa/api/user-tasks/{taskId} to read current state, task data and available completion actions. GET details can contain a form descriptor supplied by TicketFormProvider; no frontend has been implemented.

4. Approval process

4.1 Approve A as its owner

```
POST http://localhost:5000/elsa/api/user-tasks/2c67725fd185baf8/complete
Authorization: Bearer <alice-accessToken>
Content-Type: application/json

{
  "operationId": "db9f4399-c50e-4afa-9b80-8d2a3a3c3c39",
  "expectedRevision": 2,
  "data": {"comment": "Initial checks passed"},
  "actionKey": "Approve"
}
```

Captured HTTP 202. Response excerpt; fields outside the excerpt are omitted. IDs and timestamps are from an isolated test run.

```
{
  "operationId": "db9f4399-c50e-4afa-9b80-8d2a3a3c3c39",
  "status": "accepted",
  "revision": 3,
  "task": {
    "id": "2c67725fd185baf8",
    "status": "Completing",
    "revision": 3,
    "assignee": {"id": "alice"}
  }
}
```

actionKey is case-sensitive: Approve or Reject for review tasks, Resubmit for correction tasks. data is the form payload. During review, only string reason/comment fields are allowed; data={} is valid for approval. Reject requires a non-empty reason.

An accepted completion can return HTTP 202 with status=accepted and task.status=Completing while Elsa resumes the workflow. Poll the task or instance and list the next stage task. A successful acceptance is not yet proof that the next stage was created.

Current stage	After Approve	Next actor
A	B bucket	Bella or Bob claims and approves
B, amount <= 5000	D direct assignment	Dan
B, amount > 5000	C direct assignment	Clara
C	D direct assignment	Dan
D	Terminal Completed	No further task

Use the same complete endpoint with each next task ID, its latest revision and its owner token. Each stage visit creates a separate task.

4.2 Confirm completion

```
GET http://localhost:5000/elsa/api/workflow-instances/aa7aaaa3c939dc4f
Authorization: Bearer <admin-accessToken>
```

Captured HTTP 200. Response excerpt; fields outside the excerpt are omitted. IDs and timestamps are from an isolated test run.

```
{
  "id": "aa7aaaa3c939dc4f",
  "status": "Finished",
  "workflowState": {"output": {"ticketStatus": "Completed", "configurationVersion": 1}}
}
```

5. Rejection process

Rejected at	Destination	Effect
A	Requester correction	Requester edits attributes and resubmits to A.
B	A	A new A task is created. Earlier A/B decisions remain in history.
C	$rejected	Workflow ends; output.ticketStatus is Rejected.
D	Requester correction	Requester edits attributes and resubmits directly to D.

5.1 Example: Clara rejects C terminally

```
POST http://localhost:5000/elsa/api/user-tasks/1925851f2a597547/complete
Authorization: Bearer <clara-accessToken>
Content-Type: application/json

{
  "operationId": "97898eb4-6cad-4e84-859c-df3a0ee60a79",
  "expectedRevision": 1,
  "data": {"reason": "High-value claim declined"},
  "actionKey": "Reject"
}
```

Captured HTTP 202. Response excerpt; fields outside the excerpt are omitted. IDs and timestamps are from an isolated test run.

```
{
  "operationId": "97898eb4-6cad-4e84-859c-df3a0ee60a79",
  "status": "accepted",
  "revision": 2,
  "task": {
    "id": "1925851f2a597547",
    "status": "Completing",
    "revision": 2,
    "assignee": {"id": "clara"}
  }
}
```

Use each stage's own task ID and owner token. The client sends the action and reason; it does not send a destination. The interpreter reads rejectTo from the configuration snapshot stored when the workflow began.

5.2 Confirm business rejection

```
GET http://localhost:5000/elsa/api/workflow-instances/e5d42457510b71d7
Authorization: Bearer <admin-accessToken>
```

Captured HTTP 200. Response excerpt; fields outside the excerpt are omitted. IDs and timestamps are from an isolated test run.

```
{
  "id": "e5d42457510b71d7",
  "status": "Finished",
  "workflowState": {"output": {"ticketStatus": "Rejected", "configurationVersion": 1}}
}
```

Both approved and rejected business outcomes have Elsa status=Finished. Inspect workflowState.output.ticketStatus to distinguish Completed from Rejected. Rejected is a business outcome, not an Elsa execution fault.

6. Return, correct and resubmit

6.1 Alice returns A to the requester

```
POST http://localhost:5000/elsa/api/user-tasks/65e649e8839cdbc2/complete
Authorization: Bearer <alice-accessToken>
Content-Type: application/json

{
  "operationId": "347f9782-b281-46f6-9815-c400c4a917cf",
  "expectedRevision": 2,
  "data": {"reason": "Please correct the claim amount"},
  "actionKey": "Reject"
}
```

Captured HTTP 202. Response excerpt; fields outside the excerpt are omitted. IDs and timestamps are from an isolated test run.

```
{
  "operationId": "347f9782-b281-46f6-9815-c400c4a917cf",
  "status": "accepted",
  "revision": 3,
  "task": {
    "id": "65e649e8839cdbc2",
    "status": "Completing",
    "revision": 3,
    "assignee": {"id": "alice"}
  }
}
```

Log in as requester and retrieve the assigned Correct and resubmit Expense Claim task. Use Resubmit and supply the complete attributes object.

6.2 Requester changes 6000 to 4000

```
POST http://localhost:5000/elsa/api/user-tasks/fe1f5ac134421f45/complete
Authorization: Bearer <requester-accessToken>
Content-Type: application/json

{
  "operationId": "9f33ab33-2345-442e-a221-325aa0eb4854",
  "expectedRevision": 1,
  "data": {"attributes": {"claimAmount": 4000, "lastName": "Shah", "firstName": "Pranav"}},
  "actionKey": "Resubmit"
}
```

Captured HTTP 202. Response excerpt; fields outside the excerpt are omitted. IDs and timestamps are from an isolated test run.

```
{
  "operationId": "9f33ab33-2345-442e-a221-325aa0eb4854",
  "status": "accepted",
  "revision": 2,
  "task": {
    "id": "fe1f5ac134421f45",
    "status": "Completing",
    "revision": 2,
    "assignee": {"id": "requester"}
  }
}
```

The same instance continues: new A task, then B uses the corrected 4000 and routes to D. Earlier tasks retain their data snapshots. At D, correction resumes D even if the amount increases; B/C are not replayed.

7. Read the full journey

```
GET /elsa/api/user-tasks?scope=all&workflowInstanceId=<workflowInstanceId>&includeTotalCount=true
GET /elsa/api/user-tasks/<taskId>
GET /elsa/api/user-tasks/<taskId>/events
GET /elsa/api/workflow-instances/<workflowInstanceId>
GET /elsa/api/workflow-instances/<workflowInstanceId>/journal
GET /elsa/api/workflow-instances/<workflowInstanceId>/execution-state
```

Use an authorized administrator for the overview. Native task access can hide unrelated tasks with 404. Compare workflowInstanceId, reference/correlationId, task title and timestamps. Follow every task ID to read its individual event history; the definition ID identifies the template, not one claim journey.

7.1 Captured task events (first two)

```
GET http://localhost:5000/elsa/api/user-tasks/2c67725fd185baf8/events
Authorization: Bearer <alice-accessToken>
```

Captured HTTP 200. Response excerpt; fields outside the excerpt are omitted. IDs and timestamps are from an isolated test run.


```
{
  "items": [
    {
      "id": "38f5cd9775021d9d",
      "kind": "Created",
      "occurredAt": "2026-10-25T22:44:44.7020838+05:30"
    },
    {
      "id": "7926c8216cae50d5",
      "kind": "Claim",
      "occurredAt": "2026-10-25T22:44:45.0407756+05:30",
      "actorDisplayName": "alice"
    }
  ]
}
```


Task events record native lifecycle changes and actors. Task details retain the task-data snapshot; decision completion data is associated with that native task. Combine tasks, events and workflow journal for the overview. No custom consolidated journey endpoint or diagram UI has been added.

8. Errors and retries

8.1 Validation failure

```
POST http://localhost:5000/elsa/api/user-tasks/2c67725fd185baf8/complete
Authorization: Bearer <alice-accessToken>
Content-Type: application/json

{
  "operationId": "1e08551c-2454-4d85-92ea-b8fb2aa1d23b",
  "expectedRevision": 2,
  "data": {},
  "actionKey": "Reject"
}
```

Captured HTTP 422. Response excerpt; fields outside the excerpt are omitted. IDs and timestamps are from an isolated test run.

```
{"code": "form-invalid", "message": "The response did not pass validation."}
```

Status / symptom	Interpretation / next step
401	Missing, invalid or expired bearer token. Sign in again; tokens must match this server signing key.
403	Permission denied. Requester cannot publish definitions. Task membership and ownership also restrict commands.
404 on task details	Task absent or hidden from this actor. Do not assume the workflow never created it.
409	A conflicting revision or operation may require refreshing the task. A competing claimant may instead receive 403/404 depending on access evaluation.
422 form-invalid	Review/correction payload was rejected. Reject requires reason; review cannot change attributes.
200 execute with a faulted state	Activity-level business validation can become a workflow incident. Check workflowState.subStatus/incidents; do not treat HTTP 200 alone as a successful ticket submission.
202 accepted	Command accepted; poll for task completion and next work. Retry uncertain requests with the same operation ID and exact payload.
Empty task list	Check actor ID, candidate membership, scope, workflowInstanceId, state and definition version.
PostgreSQL recovery mode	Database server readiness issue. Preview can run, but it skips migrations and cannot prove persistence.

Logs include HTTP method/path/status/duration, X-Trace-Id, sanitized database target and connection/migration diagnostics, plus workflow ID, stage and visit. They do not intentionally log passwords, bearer tokens or attribute values. The host retains native scoped-service lifetime registrations required by this Elsa 3.9 sample.

The raw capture file redacts issued tokens and sample login passwords. Do not log Authorization headers in your real application. A correlation ID alone does not make workflow start idempotent; only the tested task commands use native operation IDs/revisions here.

9. Verified test results

Scenario	Result
4000 approval	A -> B -> D -> Completed
5000 approval	A -> B -> D -> Completed
6000 approval	A -> B -> C -> D -> Completed
Reject A, correct to 4000	Requester -> A -> B -> D; same workflow instance
Reject B	New A visit, then B and D
Reject C	Terminal business Rejected
Reject D, correct to 6000	Requester -> D; no automatic C re-check
Duplicate decision	Same operation ID/payload replay accepted without a second route transition
Invalid decisions	Missing rejection reason and review-time attribute edits return 422
Access / claims	Unauthenticated 401; requester publishing 403; unrelated task details 404; competing/stale claim denied
Build	dotnet build --no-restore -o bin/configresearch: passed, zero warnings
PostgreSQL restart durability	Not validated in the captured in-memory run

Repeat the checks

```
./Tests/ConfigurationApi.ps1 -BaseUrl http://localhost:5000
```

The test creates sample definitions and workflow instances in the target environment. Run it only against a disposable research instance. Full request/response bodies, apart from authentication redaction, are in docs/api-captures.json. The PDF uses explicitly labeled excerpts for readability.

Full requests and responses

See docs/api-captures.json for complete captured exchanges and Tickets.http for reusable requests. The publication body is Samples/Definitions/expense-claim.definition.json. The PDF uses labeled response excerpts for readability; dynamic IDs, revisions and timestamps differ on your server.